

csm-analysis

Architecture, decisions, and change log · started 2026-09-30

This is the single developer document for `csm-analysis`. Compile it from the repository root with `typst compile --root . notes/csm_analysis.typ`; the chapters live in `notes/architecture.typ` and are not compiled on their own. The layout follows `element-driver/notes`, and `template.typ` is a copy of the one there.

- Section 1 covers where `csm-analysis` sits among the `CriticalSoilModels` repositories, its decisions (F1, F2, ...), the proposed module layout, the open questions, and the milestones.
- Section 2 records the discussion of next steps (data sets, file readers, analysis libraries, the calibration problem). It is the context behind open questions Q4–Q6 and Q9–Q12.

Decisions of `element-driver` are cited as D1, D2, ... (see `element-driver/notes/design.typ`).

Tags. **RESOLVED** decided; **PROPOSED** designed but not in code, awaiting agreement; **OPEN** not yet decided; **DEFERRED** agreed to leave for later; **TECH DEBT** style or structure, not correctness.

Contents

1	Architecture of <code>csm-analysis</code>	2
1.1	Purpose	2
1.2	Place among the repositories	2
1.3	Decisions	2
1.4	<code>csm-tensors</code>	3
1.5	What goes in <code>record_t</code>	4
1.6	Quad plot	5
1.7	Proposed module layout	5
1.8	Calibration	6
1.9	Open questions	6
1.10	Current state (review of 2026-09-30)	7
1.11	Milestones	7
1.12	Change log	8
2	Next steps: data, readers, calibration	9
2.1	Geotechnical data to compare against	9
2.2	From lab data to <code>sig(6, n)</code> and <code>eps(6, n)</code>	9
2.3	File formats and readers	9
2.4	Libraries for analysis	10
2.5	What kind of optimization	10
2.6	Proposed order	10

1 Architecture of `csm-analysis`

1.1 Purpose

`csm-analysis` post-processes the results of constitutive-model element tests. It takes stress, strain, and time histories, from any solver or from the lab, derives the usual soil-mechanics quantities, plots them, and compares model results with measurements. The comparison pieces are the building blocks of calibration.

It is written for single-element results first, starting with `element-driver` (F9), but nothing in it knows which solver produced the data. A material-point history from an FEM or MPM code (e.g. `GeoMPM.jl`) is used the same way.

1.2 Place among the repositories

Repository	Role	Depends on
<code>csm-tensors</code>	Voigt conventions, stress and strain invariants and their derivatives, stress spaces, elastic conversions, the <code>wp</code> kind. Pure functions, no state (Section 1.4).	<code>stdlib</code>
<code>critical-soil-models</code>	Soil constitutive models, solver independent.	<code>stdlib</code> , <code>csm-tensors</code>
<code>csm-analysis</code>	Post-processing, plotting, model-data comparison; later sensitivity studies and calibration (F13). The library is solver independent (F1).	<code>stdlib</code> , <code>fortplot</code> , <code>csm-tensors</code> ; dev: <code>element-driver</code> (examples)
<code>element-driver</code>	Single-element solver. A library with no files or printing (D4). Its examples plot with a small quad plotter of their own.	<code>critical-soil-models</code> , <code>stdlib</code> , <code>csm-tensors</code> ; dev: <code>fortplot</code>
<code>Incremental_Driver</code>	Frozen legacy driver.	–

The dependency arrows point one way. `csm-tensors` is the common leaf. The `csm-analysis` **library** never imports a solver or a model; it learns about results only through arrays (F2). Since F13, `element-driver` no longer depends on `csm-analysis`; `csm-analysis` uses `element-driver` as a dev-dependency, so its examples can run element tests. `fpn` does not pass dev-dependencies on, so there is no cycle.

Change from the earlier plan

`Notes/architecture_decisions.md` (parent folder) had `csm-analysis` on top of the driver and calling it in memory for calibration. With F1 the direction is reversed: the driver depends on `csm-analysis`, and calibration calls the solver through an interface that `csm-analysis` defines and the solver side implements (Section 1.8). That note should be updated to point here.

1.3 Decisions

Discussed 2026-09-30.

#	Decision	Status
F1	<code>csm-analysis</code> is solver independent and depends on no solver or model repository. <code>element-driver</code> depends on <code>csm-analysis</code> , not the other way round. <code>csm-analysis</code> must be usable with any solver that can hand over arrays or write a file.	RESOLVED
F2	Inputs are plain arrays or files. Every analysis works on arrays in memory; file readers (later, F9) only fill those arrays.	RESOLVED
F3	Data are stored with the sign convention of <code>critical-soil-models</code> and <code>element-driver</code> (D16): tension positive, compression negative, for stress and strain. Plots show the data as stored by default, so compression is negative. An optional flag	RESOLVED

#	Decision	Status
	(compression_positive = .true.) flips the plotted quantities to the geotechnical convention. The flag only affects what is drawn; the data are never changed in place.	
F4	Calibration pieces are built in csm-analysis first: putting model and data on a common axis, and residuals. Where the optimizer loop lives (csm-analysis or a separate repository) is decided once those pieces exist (Q6).	RESOLVED
F5	Voigt order (11, 22, 33, 12, 13, 23) with engineering shear strain $\gamma_{ij} = 2\varepsilon_{ij}$, as in critical-soil-models and the original driver. Confirmed 2026-09-30: calc_two_norm_tensor_strain halves the shear terms, and element-driver's Roscoe strain rows are work conjugate with it. Now one definition, in csm-tensors.	RESOLVED
F6	Same library style as element-driver (D4, D11): no global state, no stop in library code, errors returned as a stdlib_error: state_type. Plotting and file writing are the only side effects, and only in the routines named for them. In effect in all the figures (2026-09-30).	RESOLVED
F7	Toolchain aligned with element-driver: gfortran only (D13), stdlib at ../../stdlib, fortplot from lazy-fortran/fortplot, test-drive for tests, and notes in Typst with the compile hook. In effect: fpm.toml, environment.yml (env fpm, shared with critical-soil-models; typst and jq for the notes hook).	RESOLVED
F8	Superseded by F13 (2026-09-30). Was: csm-analysis is a regular dependency of element-driver, whose examples plot with csm-analysis.	RESOLVED
F9	element-driver is the first data source, handed over in memory. Text files (CSV output, legacy Incremental_Driver files, lab data) are left for later (was Q4, Q5).	RESOLVED
F10	Invariants and Voigt helpers are implemented once, in csm-tensors, which critical-soil-models, element-driver, and csm-analysis all depend on (was Q3; Section 1.4). The name leaves room for more tensor code later.	RESOLVED
F11	No record_t for now (was Q2, option A of Section 1.5). Plot objects take arrays one test at a time. A record type shaped like element-driver's history_t would overfit csm-analysis to one solver's output format.	RESOLVED
F12	Quad plot (quad_plot_t, Section 1.6): q vs ε_q , q vs p , ε_v vs ε_q , ε_v vs p on a 2×2 grid, with the invariant pair $q = \sqrt{3J_2}$, $\varepsilon_q = \sqrt{2/3} \mathbf{e} : \mathbf{e}$ (changed from the signed triaxial pair on 2026-09-30: frame independent and defined for any path). ε_q is the default strain axis; axial strain is an option for lab data ($\varepsilon_a = \varepsilon_q + \varepsilon_v/3$ in triaxial compression, taken compression positive; equal only without volume change).	RESOLVED
F13	Renamed fumat to csm-analysis (2026-09-30) , and widened its scope: plotting, data analysis, parameter sensitivity studies, and calibration all live here, instead of a separate calibration repository; it stays usable for FEM, MPM, and element-driver data. The library remains solver independent (F1). The dependency direction is reversed from F8: element-driver no longer depends on csm-analysis (its examples have their own small quad plotter), and csm-analysis takes element-driver as a dev-dependency for its examples (example/triaxial_mcsc.f90). Package and top-level module renamed (csm-analysis, use csm_analysis); the internal fm_ module prefix is kept for now. GitHub repository renamed to CriticalSoilModels/csm-analysis and the git remote updated (2026-09-30).	RESOLVED

1.4 csm-tensors

RESOLVED Created 2026-09-30 (milestone 0) at ../../csm-tensors; not yet on GitHub.

From	Moved into csm-tensors
critical-soil-models	mod_csm_kinds (wp), src/invariants/ (values and derivatives), mod_voigt_utils, mod_voigt_conventions, mod_elastic_utils, and the 13 invariant tests with their reference implementations. Module names unchanged, so no use statement in critical-soil-models changed.
element-driver	mod_ed_stress_spaces, renamed mod_stress_spaces, with its 2 tests. Its duplicates of calc_p_inv and calc_eps_vol_inv were dropped in favour of the critical-soil-models ones (same formulas). mod_ed_kinds now takes wp from mod_csm_kinds.
new	mod_invariant_histories: calc_p_history(sig), calc_q_history, calc_lode_history, calc_eps_vol_history, calc_eps_q_history, calc_q_triax_history, calc_eps_q_triax_history, each for sig(6, n) or eps(6, n), with 3 tests.

Results: csm-tensors 18 tests, critical-soil-models 34 (was 47), element-driver 34 (was 36), all passing. The element-driver example plots are byte-identical to those from before the move. The two meanings of q keep separate names: $\text{calc_q_inv} (\sqrt{3J_2} \geq 0)$ and calc_q_triax (signed).

Elastic conversions in csm-tensors. Moved, as agreed.

- For: they are general elasticity, not soil models. csm-analysis needs them without depending on critical-soil-models (F1), for example to draw the elastic slope $3G$ on a q - ε_q plot or to seed calibration from E and ν . element-driver's linear elastic model could use them too.
- Against: the name says tensors, and $K(G, \nu)$ is scalar algebra, so the scope is less sharp. Elasticity has open-ended scope (anisotropic, pressure-dependent $G(p)$), and those models belong in critical-soil-models. The line to hold: csm-tensors has isotropic conversions and the constant stiffness matrix only.

Not done yet. The test helpers `tensor_value_check.f90` and `tensor_NaN_check.f90` are now copied in both critical-soil-models/test and csm-tensors/test. The move did not keep git history; the old files are in critical-soil-models before this change and element-driver commit 0cbb300.

1.5 What goes in record_t

RESOLVED Option A chosen (F11). With F9 there are no file readers yet, so the question is only what csm-analysis needs to plot and compare element-driver results.

What one test consists of.

Field	Need	Notes
sig(6, n), eps(6, n)	required	Everything plotted is derived from these (with the invariants library).
time(n)	usually	Creep and relaxation plots, rate-dependent models. element-driver has t_total.
label	for plots	Legend entry, e.g. "p0 = 100 kPa".
step(n)	useful	Select one stage, e.g. only the shearing after consolidation. element-driver has step and increment.
state variables	later	Plastic strain, hardening variables, void ratio. Model specific, so they would need names.
measured extras	later	Pore pressure, void ratio, cell pressure; only from lab files (F9).

element-driver's history_t also has n_iter (Newton iterations), which is solver diagnostics rather than test data.

Option A: no record type yet; the figure holds the data. Plot objects take arrays one test at a time and keep what they need:

```

type(triaxial_plot_t) :: fig
do i = 1, 3
  call fig%add(history(i)%sig, history(i)%eps, label=labels(i))
end do
call fig%save("output/triaxial_mcscs.png", compression_positive=.true.)

```

This keeps the whole API array based (F2), and element-driver's core types stay free of csm-analysis types. The cost: each analysis routine repeats the argument list (sig, eps, time, ...), and a record type still arrives with the file readers.

Option B: a small record_t now. label, time, sig, eps, and optional step, built from arrays by a constructor. Plots and comparisons take record_t. Variant B2: element-driver's history_t extends record_t (adding increment, n_iter), so a history is passed to csm-analysis with no copy and no conversion. B2 ties element-driver's core result type to csm-analysis, so a change to record_t changes the driver's API.

Decision: option A (F11). A record type is revisited only with the file readers, and then shaped by the files, not by history_t.

1.6 Quad plot

RESOLVED In code 2026-09-30: src/fm_quad_plot.f90, re-exported by use csm_analysis.

```

type(quad_plot_t) :: fig
do i = 1, 3
  call fig%add(history(i)%sig, history(i)%eps, label=labels(i))
end do
fig%compression_positive = .true. ! F3; default .false.
call fig%save("output/triaxial_mcscs.png", status)

```

- **Variables.** Invariants only, from csm-tensors (mod_invariant_histories): $p = \text{tr}(\sigma)/3$, $q = \sqrt{3J_2} \geq 0$, $\varepsilon_v = \text{tr}(\varepsilon)$, $\varepsilon_q = \sqrt{2/3} e : e \geq 0$ (shear terms halved, F5). They do not depend on the axes of the test, so the plot also works for simple shear or true triaxial paths. $q d\varepsilon_q$ equals the deviatoric work $s : de$ when the strain increment is coaxial with s , as in any triaxial test; in general it does not. In a triaxial test $q = |\sigma_a - \sigma_r|$ and $\varepsilon_q = \frac{2}{3} |\varepsilon_a - \varepsilon_r|$, the sizes of the signed Roscoe variables. The signed pair (calc_q_triax, calc_eps_q_triax) stays in csm-tensors for element-driver's constraint rows but is not plotted.
- **Signs.** q and ε_q are magnitudes and never flipped; compression_positive flips p , ε_v , and the optional axial strain. Triaxial compression and extension therefore look the same in q ; the Lode angle tells them apart (not plotted).
- **Options** (components, set before save): compression_positive, strain_in_percent (default true), use_axial_strain, title, stress_unit (default kPa), width, height.
- **Errors.** add has no status; the first shape error is kept and returned by save as a state_type (F6).
- **Legend workaround (Q7).** Each test gets a fixed colour from the matplotlib tab10 cycle, and the colour key (blue: 50 kPa orange: 100 kPa ...) is the title of the top-left panel. subtitle is one line only, and a long key is clipped at the panel width, so labels should be short.
- **Tests.** 5 in test/test_fm_quad_plot_suite.f90 (errors, PNG written with default and all options). The plots themselves were checked by eye: example/quad_plot_synthetic.f90 and the element-driver examples triaxial_linear_elastic (matches $q = E\varepsilon_a$, $\varepsilon_v = (1 - 2\nu)\varepsilon_a$) and triaxial_mcscs.

1.7 Proposed module layout

PROPOSED Names follow element-driver (D14): a wp kind, sig and eps as $(6, n)$ arrays, and a mod_fm_ prefix for modules, like mod_ed_ there.

Module	Contents
mod_fm_quad_plot	RESOLVED The triaxial quad plot (Section 1.6).

Module	Contents
mod_fm_kinds	RESOLVED wp from csm-tensors.
mod_fm_oedometer_plot	RESOLVED oedometer_plot_t: σ_v vs ε_a and σ_h vs σ_v side by side, axis 1 vertical, $\sigma_h = (\sigma_{22} + \sigma_{33})/2$. All signed, so compression_positive flips everything. $e\text{-log } \sigma'_v$ needs a void ratio and a log axis; not done.
mod_fm_xy_plot	RESOLVED xy_plot_t: any curves on one set of axes; the caller computes and scales x and y . On a single set of axes fortplot's legend works, so it is used.
mod_fm_plot_style	RESOLVED Shared by the figures: tab10 colour cycle, colour key, sign and strain scaling, shape check, panel labels.
mod_fm_compare	DEFERRED Model-data comparison: interpolate the model onto the measured abscissa (e.g. axial strain), residual vectors, weights, and normalization per quantity.
mod_fm_record, mod_fm_csv, mod_fm_lab	DEFERRED With the text files (F9).

Invariants come from csm-tensors (mod_invariant_histories), not from a csm-analysis module. element-driver's examples then call mod_fm_plot instead of carrying their own p_of, q_of, eps_v_of helpers and sign flips.

1.8 Calibration

DEFERRED Recorded so that F1 does not block it later.

A calibration loop needs to run the solver many times, but by F1 csm-analysis cannot call element-driver. The usual way round this is an abstract type in csm-analysis, extended on the solver side:

```
type, abstract :: simulator_t
contains
  procedure(simulate_iface), deferred :: simulate
    ! simulate(self, params, results, status): run every test for one parameter set
end type
```

The optimizer (e.g. modern-minpack's Levenberg-Marquardt, as planned in Notes/architecture_decisions.md) sees only simulator_t, the measured data, and mod_fm_compare. An element-driver simulator lives in element-driver or in the user's program. Because element-driver has no global state (D11), parameter sets can run in parallel. The kind of optimization problem and the candidate optimizers are discussed in Section 2.5.

1.9 Open questions

#	Question	Status
Q1	Regular or dev-dependency? Regular (F8).	RESOLVED
Q2	What goes in record_t? Not needed now (F11).	RESOLVED
Q3	Shared invariants library? csm-tensors (F10).	RESOLVED
Q4	CSV reader (stdlib_io: loadtxt or csv-fortran). loadtxt is numeric only; mapping columns by name needs csv-fortran or a sidecar (Section 2.3).	DEFERRED
Q5	Which lab data first. Karlsruhe fine sand suggested; candidates in Section 2.1.	OPEN
Q6	Where the optimizer lives (F4), once mod_fm_compare exists.	DEFERRED

#	Question	Status
Q7	fortplot subplot legend. Checked 2026-09-30 against lazy-fortran/fortplot af5f26e: figure_t%legend draws all entries on top of each other in the lower-left corner of the figure and ignores location; supitle does not break lines; figure_t has no text call. Worked around in quad_plot_t (Section 1.6). Reporting upstream is still open.	OPEN
Q8	Invariant or triaxial pair in the quad plot? Invariant pair (F12, 2026-09-30).	RESOLVED
Q9	Metadata sidecar: JSON or TOML? Units, sign convention, column map, initial state (Section 2.3).	OPEN
Q10	Which model is calibrated first? Its parameters decide which tests matter (Section 2.6).	OPEN
Q11	Undrained tests (pore pressure) in scope from the start? Affects mod_fm_lab and the residuals (Section 2.2).	OPEN
Q12	Optimization strategy and libraries. LM (minpack), BOBYQA (PRIMA), global search (differential evolution, pikaia); residual abscissa and normalization (Section 2.5).	OPEN

1.10 Current state (review of 2026-09-30)

Item	Finding	Tag
Code	src/fumat.f90 is a hello-world. app/main.f90 plots $\sin^2 x$ with fortplot and writes PNGs. Builds and runs with the fpm conda env (fpm 0.13, gfortran 15.2).	–
fpm.toml	fortplot points at krystophny/fortplot, pinned in the build cache to a 2025 revision; element-driver uses lazy-fortran/fortplot.stdlib = "*" (registry), while element-driver uses ../../stdlib.	TECH DEBT
Dependency check	A scratch copy with element-driver as a path dependency, lazy-fortran fortplot, and ../../stdlib built and linked. The root's stdlib overrode critical-soil-models' git one without conflict. (That tested the direction F1 rules out, but it shows the stdlib and fortplot choices of F7 work together.)	–
Repository	src/fumat.mod is committed; there is no .gitignore. The untracked text.png, test_100.png, and text.pdf are outputs of app/main.f90.	TECH DEBT
Docs	The README names environments.yml (the file is environment.yml) and says csm-analysis analyzes incremental-driver output. environment.yml installs lfortran (not supported, D13). .vscode/settings.json uses the env fpm, while the README creates csm.	TECH DEBT

1.11 Milestones

#	Scope
0	RESOLVED csm-tensors (F10). Repository created, code moved from critical-soil-models and element-driver, history versions added, both repositories switched, all tests passing.
1	RESOLVED Clean-up. .gitignore, fumat.mod and the stale fpm.rsp removed, fpm.toml aligned (F7), test-drive set up, app/main.f90 moved to example/fortplot_basics.f90, README and environment.yml rewritten.

#	Scope
2	RESOLVED Plots. Quad plot (Section 1.6), oedometer plot, x - y overlay; 11 tests. <code>element-driver</code> depends on <code>csm-analysis</code> (F8), all three of its examples plot with it, and it no longer uses <code>fortplot</code> directly. Still open: report the legend bug upstream (Q7).
3	Comparison. <code>mod_fm_compare</code> : interpolation and residuals, tested with one <code>element-driver</code> run as synthetic “data”. Then decide Q6 and the <code>simulator_t</code> interface.
–	Deferred. Text files: CSV, legacy output, lab data, <code>record_t</code> readers (F9). Proposed order for this and milestone 3 in Section 2.6: data set, readers, comparison, optimizer.

1.12 Change log

- 2026-09-30: notes started; decisions F1–F4 agreed; F5–F7 and the layout proposed.
- 2026-09-30: F8 (regular dependency), F9 (`element-driver` first, text files later), and F10 (shared invariants library) agreed. Sections on the shared library and on `record_t` added. Milestones renumbered.
- 2026-09-30: F5 confirmed; F10 named `csm-tensors`; F11 (no `record_t`, option A). Milestone 0 done: `csm-tensors` created and used by `critical-soil-models` and `element-driver`.
- 2026-09-30: F12 and the quad plot (`quad_plot_t`); Q7 checked and worked around; Q8 added. Milestone 1 mostly done, milestone 2 started. `element-driver` now depends on `csm-analysis`.
- 2026-09-30: The quad plot uses the invariant pair $q = \sqrt{3J_2}$, ε_q instead of the signed triaxial pair (F12, Q8).
- 2026-09-30: Milestones 1 and 2 done: `oedometer_plot_t`, `xy_plot_t`, shared `mod_fm_plot_style`; README and `environment.yml`; F6, F7 in effect. 11 tests pass.
- 2026-09-30: Next steps discussed (Section 2): candidate data sets, lab data to Voigt arrays, CSV plus metadata sidecar, reader and analysis libraries, the calibration problem, a proposed order. Q4 and Q5 updated; Q9–Q12 added. Nothing decided.
- 2026-09-30: Renamed to `csm-analysis` (F13; F8 superseded). `element-driver` dropped its dependency; `element-driver` is a dev-dependency here, with `example/triaxial_mcsc.f90` (MCSS drained triaxial at three cell pressures, run by `element-driver`, plotted with `quad_plot_t`). 11 tests pass.
- 2026-09-30: Remaining `fumat` references in code, README, CLAUDE.md, and notes changed to `csm-analysis`; the notes’ quad-plot section now says use `csm-analysis`. The name is kept only in F13 and in the 2026-09-30 review of the old state. GitHub repository renamed to `CriticalSoilModels/csm-analysis`; the git remote points there.

2 Next steps: data, readers, calibration

PROPOSED Discussed 2026-09-30, after milestone 2. Nothing in this chapter is agreed yet; it records the options so that the decisions (Q4, Q5, Q6, Q9–Q12) can be made later with the context at hand.

The deferred pieces of Section 1 (file readers F9/F11, `mod_fm_compare`, calibration Section 1.8) depend on each other in a fixed order: **data, then readers, then comparison, then optimization**. The data set chosen first shapes the file format, the metadata, and which residuals matter, so it comes first.

2.1 Geotechnical data to compare against

OPEN Q5. Candidates named in the discussion. The links and current availability have **not** been checked yet.

Source	Why it is a candidate
Karlsruhe fine sand (Wichtmann and Triantafyllidis)	Leading candidate. Open, one sand, many drained and undrained monotonic triaxial tests, oedometric and cyclic tests, published for model calibration (hypoplasticity, SANISAND, ...). Plain text or Excel.
DesignSafe-CI Data Depot (NHERI), LEAP projects	Ottawa F-65 sand: cyclic triaxial, DSS, and monotonic tests from several labs. Well documented, but each lab has its own format.
Verdugo and Ishihara (1996), Toyoura sand	The classic critical-state check: drained and undrained tests over a range of densities. Usually digitized from the paper, so lower quality, but almost every model paper compares with it.
Zenodo, Mendeley Data	More and more calibration papers deposit their raw lab data here. Search “triaxial” with the name of the sand.
AGS4 (UK), DIGGS (US, XML)	Exchange formats for site-investigation data. Not worth supporting now; useful as references when designing our own metadata (Section 2.3).

2.2 From lab data to `sig(6, n)` and `eps(6, n)`

Lab files are not in `csm-analysis`’s form (F2, F3, F5). A triaxial test gives σ_a, σ_r (or q, p'), $\varepsilon_a, \varepsilon_v$, often pore pressure u ; compression positive; strains often in % and stresses in kPa. `mod_fm_lab` would

- convert units and flip signs to tension positive (F3);
- fill the Voigt arrays under axisymmetry, axis 1 vertical: $\sigma_{22} = \sigma_{33} = \sigma_r, \varepsilon_{22} = \varepsilon_{33} = (\varepsilon_v - \varepsilon_a)/2$, shear terms zero;
- keep the extras: initial void ratio e_0 , consolidation stress, drainage condition, u .

`element-driver` needs those extras to **replay** the test (initial state and loading path), which is where `record_t` (F11) comes back: it is to be shaped by the lab files, as F11 says, not by `history_t`.

2.3 File formats and readers

OPEN Q4, Q9. Suggested layout: **a CSV with the numbers plus a small sidecar file with the metadata** (test type, units, sign convention, which column holds which quantity, initial state).

Need	Library	Notes
numeric CSV or whitespace	<code>stdlib_io loadtxt, savetxt</code>	Already a dependency. In <code>./././stdlib</code> (9e4c2302, 2025-10-17) <code>loadtxt</code> takes <code>skiprows</code> , <code>max_rows</code> , and a one-character

Need	Library	Notes
		delimiter (checked). Numbers only: no column names.
CSV with headers or mixed types	jacobwilliams/csv-fortran	Columns mapped by header name.
JSON	jacobwilliams/json-fortran	Mature, builds with fpm.
TOML	toml-f	What fpm itself uses; good for calibration run files.
.npy	stdlib_io_npy	Already available; easy exchange with Python.
HDF5, netCDF	–	Too heavy for element tests; not now.

2.4 Libraries for analysis

- **stdlib** (already a dependency): `stdlib_stats` (mean, variance, correlation, moments), `stdlib_linalg` (least squares, SVD; for sensitivity and covariance at the optimum), sorting, strings.
- **Interpolation** for `mod_fm_compare`: `jacobwilliams/finterp` (linear) or `jacobwilliams/bspline-fortran`.
- **Optimizers**:
 - `fortran-lang/minpack`: Levenberg–Marquardt, already planned (Section 1.8);
 - **PRIMA** (Zaikun Zhang): modern Fortran BOBYQA, NEWUOA, COBYLA; derivative free, with bounds. Strongly worth considering (see Section 2.5);
 - global search: `jacobwilliams/pikaia` (genetic algorithm) or a small differential evolution written here.

2.5 What kind of optimization

OPEN Q6, Q12. Calibration is **bounded, weighted, nonlinear least-squares parameter identification over several tests**. Features of constitutive models that steer the method:

- **Noisy, non-smooth objective**. Yield-surface switches and substepping tolerances make finite-difference gradients unreliable. This favours a derivative-free local method (BOBYQA) over plain LM, or LM with carefully chosen steps.
- **Several minima, correlated parameters** (e.g. λ and N trade off). Usual recipe: global search (differential evolution or CMA-ES), then local polish, then the Jacobian at the optimum to report sensitivity and identifiability.
- **Residual design, the hard part, in `mod_fm_compare`**.
 - Abscissa: ε_a for monotonic tests; arc length or step index for cyclic or non-monotonic paths.
 - Normalization per quantity: q in kPa and ε_v dimensionless must be made comparable.
 - Weights per test.
- **Parameter scaling**. Optimize in normalized or log space; parameters span orders of magnitude.
- **Later**: multi-objective (Pareto front between, say, triaxial and oedometer fits), Bayesian or MCMC for uncertainty. Element tests are cheap, so these are feasible.

Element tests run fast and `element-driver` has no global state (D11), so a population of parameter sets can run in parallel (OpenMP), which is an advantage of doing this in Fortran.

2.6 Proposed order

1. Choose one reference data set (Karlsruhe fine sand suggested) and one model to calibrate (Q5, Q10).
2. `mod_fm_lab`, `mod_fm_csv`: CSV plus sidecar into Voigt arrays and initial state; revisit `record_t` (F11, Q4, Q9).
3. `mod_fm_compare`: interpolation, residuals, normalization; check by plotting model against data with the existing figures (milestone 3).
4. `simulator_t`, then LM (minpack) and BOBYQA (PRIMA) on one test, then several tests, then a global search (Q6, Q12).